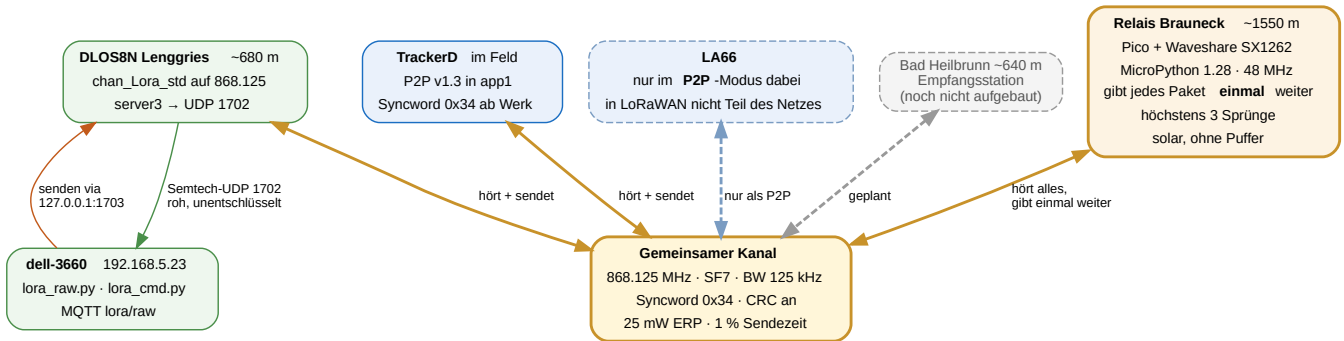


Relaisstelle Brauneck

Krisennetz Lenggries — Aufbau, Betrieb und Befehle. Stand 16.08.2026 · Repository `gerontec/TTN`



1 Grundgedanke: ein gemeinsamer Kanal

Alle Teilnehmer arbeiten auf **einer** Frequenz mit **einem** Spreizfaktor. Das ist keine Sparmaßnahme, sondern zwingend: Ein einzelnes Funkmodul kann immer nur auf einem Kanal lauschen. Nur so hört das Relais jeden und erreicht jeden. Es ist dasselbe Flutungsverfahren, das Meshtastic und Ebytes Broadcast-Modus benutzen.

Parameter	Wert	Warum
Frequenz	868.125 MHz	Werksvorgabe der TrackerD-P2P-Firmware; liegt im Band 868.0–868.6
Spreizfaktor	SF7	reicht: 30 dB Reserve auf 10 km Sichtverbindung
Bandbreite	125 kHz	Standard
Syncword	0x34 (öffentlich)	Der SX1302 im Gateway kennt nur <i>ein</i> Syncword für den ganzen Chip. Ein Node auf dem privaten 0x12 wird nicht gehört.
CRC	an	sonst verwirft der Paket-Forwarder
Leistung	bis 14 dBm ERP	25 mW ist das Limit in 868.0–868.6
Sendezeit	1 %	bei ~72 ms Luftzeit rund 7 s Sperre je Paket

Streckenrechnung 10 km, Sichtverbindung. 14 dBm + 2 dBi = 16 dBm EIRP, minus 111 dB Freiraumdämpfung, plus 2 dBi Empfangsantenne ergibt –93 dBm. Gegen –123 dBm Empfindlichkeit bei SF7 bleiben rund **30 dB Reserve**. Den Sprung ins Nachbartal trägt der Bergstandort (~1550 m gegen zwei Täler auf ~650 m), nicht die Sendeleistung.

2 Was das Relais tut

Es hört alles auf dem gemeinsamen Kanal und gibt jedes Paket **genau einmal** weiter, versehen mit einem Sprungzähler. Damit erreicht ein Knoten, dessen Direktsignal zu schwach wäre, das Tal trotzdem.

Nutzlast beginnt mit	Bedeutung	Verhalten
C>	Fernwirkbefehl	ausgeführt, Antwort gesendet, nie weitergegeben
A>	Antwort einer Station	ignoriert
R<n>>	bereits n-mal weitergegeben	weiter bis 3 Sprünge, danach verworfen
alles Übrige	frisches Paket	als R1>... weitergegeben

Eigenecho

Da alle denselben Kanal teilen, trägt allein die Marker-Logik:

- Während des Sendens ist der Empfänger taub — die eigene Aussendung hört die Station nie unmittelbar.
- Jedes weitergegebene Paket bekommt R<sprung>>. Kommt es über eine zweite Relaisstelle zurück, greift der Zähler.
- Der Dublettenspeicher schlüsselt auf den Inhalt **ohne** Marker und sperrt denselben Text fünf Minuten.

Mehrere Relaisstellen sind dadurch möglich: aus R1> wird R2> und so fort, bis drei Sprünge erreicht sind.

3 Fernwirken von 192.168.5.23

Der Pico hat **kein WLAN** — es ist ein RP2040, kein Pico W; das Modul `network` existiert nicht. Auf dem Berg gibt es weder SSH noch OTA. Der einzige Rückkanal ist der Funk, auf dem das Relais ohnehin arbeitet. Für ein Krisensystem ist das der richtige Weg: Er trägt genau dann, wenn alles andere ausgefallen ist.

```
python3 /home/gh/python/lora_cmd.py POWER 17
gesendet: C>POWER 17
Antwort: POWER 17 dBm (RSSI -101, SNR 8.8)
```

Befehl	Wirkung
POWER 2..22	Sendeleistung in dBm — der wichtigste Befehl
STATUS	weitergegeben / unterdrückt / Laufzeit / Konfiguration
RELAY 0 1	Weitergabe aus- oder einschalten
TELEM 0 1	Quittungen aus- oder einschalten
SAVE	Konfiguration ausdrücklich sichern
REBOOT	Neustart des Boards
PING	lebt die Station?

Frequenz und Spreizfaktor sind bewusst nicht fernstellbar. In einem Einkanalnetz würde man sich damit den Ast absägen, auf dem man sitzt — die Station wäre nach dem Wechsel unerreichbar.

Ohne Authentisierung. Wer in Funkreichweite ist, kann das Relais umstellen. Bewusste Abwägung zugunsten der Einfachheit.

Weg des Befehls: `lora-raw.service` hält UDP 1702 dauerhaft und ist der einzige, der senden kann. Er hat deshalb einen Steuereingang auf `127.0.0.1:1703` ; was dort ankommt, wird beim nächsten `PULL_DATA` des Gateways gefunkt. `lora_cmd.py` schickt den Befehl dorthin und wartet über MQTT `lora/raw` auf die Antwort.

4 Betriebsart umschalten

TrackerD — LoRaWAN ↔ P2P

Der TrackerD hat zwei Firmware-Slots: **app0** trägt die Dragino-LoRaWAN-Firmware, **app1** die selbst gebaute P2P-Firmware. Umgeschaltet wird ausschließlich über `otadata` — `app0`, `app1` und vor allem das NVS mit den LoRaWAN-Keys bleiben unangetastet. Der Wechsel ist daher verlustfrei und beliebig wiederholbar.

Richtung	Befehl	gemessen
LoRaWAN → P2P	<code>switch_app.py p2p --port /dev/ttyACM1</code>	1,35 s
P2P → LoRaWAN	<code>AT+LORAWAN</code> — am Gerät, ohne PC-Werkzeug	2,24 s inkl. Neustart
P2P → LoRaWAN	<code>switch_app.py lorawan --port /dev/ttyACM1</code>	gleichwertig
Zustand abfragen	<code>switch_app.py status --port /dev/ttyACM1</code>	liest <code>otadata</code>
P2P neu bauen	<code>pio run</code> , dann <code>switch_app.py flash</code>	schreibt nur <code>app1</code>

Niemals `pio run -t upload`. Das würde Partitionstabelle und `app0` überschreiben und die LoRaWAN-Firmware samt Keys zerstören. Geflasht wird gezielt nach `0x1F0000` durch `switch_app.py`.

LA66 — LoRaWAN ↔ P2P

```
python3 la66_mode.py status      # Modus am Boot-Banner erkennen
python3 la66_mode.py p2p         # 37640 B, 5,0 s
python3 la66_mode.py lorawan     # 69032 B, 9,1 s
```

Beim LA66 wird wirklich geflasht, nicht umgeschaltet: Beide Dragino-Images sind fest auf `0x0800D000` gelinkt, ein zweiter Slot ist unmöglich. Ein Flash setzt die Funkparameter auf Werk zurück — dafür `--apply-rf`. Die DevEUI überlebt.

Im LoRaWAN-Modus ist ein Gerät nicht Teil dieses Netzes. Nachgemessen: Der LA66 sendet mit `AT+DR=0` auf SF12 und springt über alle acht EU868-Kanäle (beobachtet: 867.3 und 868.3 MHz). Der Pico auf 868.125/SF7 hörte in 95 s **null** Pakete. Spreizfaktoren sind quasi-orthogonal — ein SF7-Empfänger kann SF12 nicht demodulieren.

Daraus folgt grundsätzlich: **Ein Knoten mit einem Funkmodul kann kein LoRaWAN-Relais sein.** LoRaWAN wechselt pro Sendung den Kanal und passt per ADR den Spreizfaktor an; ein Empfänger mit fester Frequenz und festem SF kann dem nicht folgen. Genau dafür hat das Gateway acht parallele Demodulatoren. Hinzu kommt: Der Marker `R1>` würde die MIC-Prüfung eines LoRaWAN-Rahmens zerstören.

5 Stromversorgung: reiner Solarbetrieb

Der Victron-Laderegler schaltet den Verbraucher morgens schlagartig zu und abends ab. Eine Pufferbatterie gibt es nicht. Das prägt den Betrieb stärker als jede Funkeinstellung.

- **Jeder Morgen ist ein Kaltstart, und niemand ist oben.** `main.py` startet das Relais, wiederholt bei Fehlern und löst notfalls `machine.reset()` aus, statt in den REPL zu fallen.
- **Konfiguration sichert sich sofort**, nicht erst auf `SAVE` — eine per Funk gesetzte Sendeleistung wäre sonst am nächsten Morgen weg.
- **Beschädigte `relais.json` wird abgefangen**; die Station kommt dann mit Vorgabewerten hoch, aber sie kommt hoch.
- **Nachts ist die Kette unterbrochen.** Eigenschaft des Aufbaus, keine Störung — wer nachts Reichweite braucht, braucht einen Akku.

Systemtakt 48 MHz

125 MHz sind sinnlos: Modulation, Timing und Preamble macht der SX1262 selbst, der Prozessor wartet nur. Weniger Takt heißt weniger Strom, und bei einer Versorgung ohne Puffer weniger Spannungseinbruch unter Last.

Takt	Ergebnis der Abtastung
125 / 96 / 64 / 48 / 32 / 24 MHz	sauber — <code>DevErr 0x0000</code> , Syncword <code>3444</code> , TX ok
18 MHz	SPI liefert Müll — Syncword liest <code>a2a2</code> , TX schlägt fehl
12 MHz	<code>machine.freq()</code> lehnt ab

Gewählt sind **48 MHz**: reichlich Abstand zur Ausfallgrenze bei gut halbiertem Prozessorstrom. Der Takt wird vor dem Aufsetzen des Funkchips gestellt, weil `clk_peri` am Systemtakt hängt.

6 Nachgewiesen über die Luft

Alle drei Stationen in einem Durchlauf, nach dem Kaltstart der neuen Firmware und **ohne** manuelles Nachsetzen des Syncwords:

```
TrackerD sendet  "V13-KALTSTART-1"
Pico            weiter: Sprung 1 RSSI -17 SNR 12.0 20 dBm, 51 ms
Gateway hört    RSSI -83 "V13-KALTSTART-1"      <- direkt
Gateway hört    RSSI -84 "R1>V13-KALTSTART-1"    <- über das Relais
Pico-Bilanz     2 gehört, 2 weitergegeben, 0 unterdrückt
```

In einem früheren Lauf war der Gewinn deutlicher messbar: Original -92 dBm, Kopie vom Relais -73 dBm — **19 dB stärker**. Genau dafür steht die Station auf dem Berg. Der TrackerD empfing dabei sein eigenes Paket als `R1>...` zurück; das Relais gab es **nicht** erneut weiter.

7 Dateien

Ort	Datei	Zweck
Pico	<code>main.py</code>	Selbststart nach jedem Sonnenaufgang
	<code>repeater.py</code>	Flutung, Sprungzähler, Dubletten, Sendezeitbudget
	<code>fernwirk.py</code>	Befehle, Konfiguration in <code>/relais.json</code>
	<code>lora_p2p.py</code>	SX1262-Treiber
dell 192.168.5.23	<code>lora_raw.py</code>	Roh-Abgriff UDP 1702, Steuereingang 1703, MQTT
	<code>lora_cmd.py</code>	Fernwirkbefehle absetzen
TrackerD	<code>p2p/src/main.cpp</code>	P2P-Firmware v1.3, Syncword 0x34 ab Werk
	<code>switch_app.py</code>	otadata-Umschaltung, Flashen nach app1

Erzeugt aus `devices/pico_sx1262/` im Repository gerontec/TTN. Zeichnung: `relais_uebersicht.dot`.